Advanced Architectural Blueprint for a ViZDoom Agent: Integrating Behavioral Cloning Pre-training with Proximal Policy Optimization

1. Introduction to Visual Reinforcement Learning in 3D Environments

The historical progression of Artificial Intelligence, specifically within the domain of Deep Reinforcement Learning (DRL), is inextricably linked to the complexity of the simulated environments utilized to benchmark algorithmic performance and cognitive capacity. While early foundational breakthroughs relied on the Arcade Learning Environment (ALE) and fully observable two-dimensional state spaces—demonstrating that convolutional neural networks could successfully map high-dimensional pixel inputs to discrete control policies—these flat environments fundamentally lack the geometric and distinct cognitive challenges inherent in the physical world.[1] The transition from third-person, fully observable two-dimensional state spaces to first-person, partially observable three-dimensional navigation represents a critical inflection point in the pursuit of generalized autonomous agents.[1]

The ViZDoom research platform, built upon the foundational engine of the seminal 1993 video game Doom, emerged specifically to bridge this chasm.[1] Maintained currently by the Farama Foundation, ViZDoom offers a computationally efficient yet visually complex three-dimensional world that forces agents to operate under the strict constraints of embodied perception.[1] Unlike the omniscient perspective afforded by traditional board games or top-down mazes, a ViZDoom agent perceives only the data contained within its immediate, restricted field of view. To achieve mastery in this environment, algorithms must not merely approximate a shallow function mapping pixels to actions; they must synthesize complex representations of three-dimensional space, develop object permanence, manage long-horizon temporal dependencies, and navigate the reinforcement learning challenges of instability arising from function approximation and off-policy bootstrapping.[1]

This comprehensive report provides an exhaustive technical analysis and an architectural blueprint for developing a specialized artificial intelligence agent capable of mastering Doom. The proposed architecture employs a sophisticated two-phase hybrid learning paradigm. The initial phase leverages Behavioral Cloning (BC)—a form of Imitation Learning (IL)—derived from high-fidelity human gameplay recordings to instill foundational navigational and combat heuristics.[3] The subsequent phase transitions the pre-trained neural network into an online trial-and-error fine-tuning regimen utilizing Proximal Policy Optimization (PPO).[5] By leveraging

the Stable Baselines 3 (SB3) framework alongside the imitation library, the architecture processes multi-modal inputs, including RGB screen buffers, geometric depth buffers, and semantic segmentation labels, through a custom-engineered Convolutional Neural Network (CNN) feature extractor.[6] This multi-stage methodology fundamentally mitigates the sample inefficiency inherent in cold-start reinforcement learning, establishing robust, human-like behaviors before subjecting the model to the strict, high-variance optimization topography of PPO reward signals.[4]

2. Mathematical Foundations of Reinforcement Learning

Before detailing the specific software architecture and neural network configurations, it is necessary to formally define the mathematical underpinnings of the decision-making process that governs the agent's behavior. The game of Doom, from the perspective of an autonomous agent, is natively a Partially Observable Markov Decision Process (POMDP) due to the first-person perspective. However, for the sake of standard reinforcement learning formalization and algorithmic application, it is often approximated as a standard Markov Decision Process (MDP) by stacking historical frames to simulate full state observability.[5]

2.1. The Markov Decision Process Framework

A Markov Decision Process is formally defined as a mathematical tuple consisting of five core components: $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.[10] The State Space ($\mathcal{S}$) represents the set of all possible states $s_t \in \mathcal{S}$ that the agent can occupy. In the context of ViZDoom, this is represented by continuous, multi-dimensional pixel matrices alongside discrete internal game variables such as health and ammunition.[1] The Action Space ($\mathcal{A}$) is the finite set of available actions $a_t \in \mathcal{A}$ dictated by the agent's movement and combat mechanics.[10]

The Transition Probability ($\mathcal{P}$) defines the environmental dynamics, representing the mathematical probability of transitioning to a subsequent state s' given the current state s and a chosen action a. This is expressed as $\mathcal{P}(s'|s,a) = P$.[10] The Reward Function ($\mathcal{R}$) is the scalar feedback signal received immediately after an action is executed, defined as the expected value $\mathcal{R}(s,a) = \mathbb{E}$.[10] Finally, the Discount Factor (γ), a scalar value where $\gamma \in $, weights the importance of future rewards against immediate gratification. This discount factor defines the Expected Cumulative Return G_t, which is the ultimate objective the agent seeks to maximize, calculated as the infinite sum of discounted future rewards:

$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.[10]

The fundamental objective of the agent is to learn a Policy (π), which dictates its behavior by mapping states to probabilities of selecting specific actions. A stochastic policy is mathematically defined as $\pi(a|s) = P$, whereas a purely deterministic policy maps a state directly to a single action: $a = \pi(s)$.[10]

2.2. Value Functions and the Bellman Optimality Operator

To optimize the policy, the architecture relies on accurately estimating the expected return from any given state. The State-Value Function, denoted as $V^{\pi}(s)$, quantifies the expected cumulative return when an agent begins in state s and follows policy π indefinitely.[10] This is expressed as $V^{\pi}(s) = \mathbb{E}_{\pi}$.[10] Similarly, the Action-Value Function, $Q^{\pi}(s, a)$, calculates the expected return of taking a specific action a in state s, and subsequently adhering to policy π. This function is critical for evaluating the immediate impact of deviations from the policy, defined as $Q^{\pi}(s, a) = \mathbb{E}_{\pi}$.[10]

The pursuit of the optimal policy π^* requires finding the optimal action-value function $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$.[10] If the optimal Q-function is known, the optimal policy can be trivially derived by selecting the action that maximizes the Q-value at any given state. The theoretical foundation for discovering this optimal function relies on the Bellman Optimality Equation, a non-linear system of equations whose unique fixed point is Q^*.[10] The Bellman Optimality Operator $\mathcal{T}^*$ acts upon Q-functions such that $Q^* = \mathcal{T}^* Q^*$. In environments with finite state spaces, this operator is proven to be a contraction mapping under the infinity norm, guaranteeing convergence.[10]

2.3. The Limitations of Deep Q-Networks in 3D Environments

While classic algorithms like Q-learning approximate the fixed point of the Bellman operator using temporal difference updates, applying these methods to the massive, continuous state spaces of 3D gaming necessitates function approximation via deep neural networks, resulting

in the Deep Q-Network (DQN) algorithm.[1] However, simply replacing a discrete Q-table with a neural network introduces severe theoretical and practical instabilities.[10]

The primary issue is the presence of non-stationary targets. In DQN, the Bellman target $y_t = R_{t+1} + \gamma \max_{a'} Q_\theta(S_{t+1}, a')$ depends directly on the network parameters θ. Consequently, every gradient update shifts the target the network is attempting to learn, leading to volatile bootstrapping and potential divergence.[10] Furthermore, sequential states generated by an agent exploring an environment are highly temporally correlated, violating the independent and identically distributed (i.i.d.) assumptions required for stochastic gradient descent (SGD) to function optimally.[10]

To mitigate these issues, DQN implementations typically require a frozen Target Network (Q_{θ^-}) that updates only periodically, alongside a massive Experience Replay memory buffer that stores millions of transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ to break temporal correlations through random minibatch sampling.[10] Despite these architectural additions, off-policy value iteration methods often struggle with the "Deadly Triad" of reinforcement learning in environments like ViZDoom, exhibiting high sensitivity to hyperparameters and prolonged training times compared to on-policy alternatives.[1]

2.4. Policy Gradients and Variance Reduction

Given the limitations of action-value approximation in high-dimensional continuous observation spaces, modern architectures heavily favor Policy Gradient methods. Rather than attempting to learn a Q-function and deriving a greedy policy indirectly, policy gradient algorithms directly parameterize the policy itself, $\pi_\theta(a|s)$, using a deep neural network where θ represents the network weights.[10] The objective is to directly maximize the expected return over all possible trajectories τ generated by the policy: $J(\theta) = \mathbb{E}_{\tau \sim P_\theta}[G(\tau)]$.[10]

The analytical gradient of this objective, foundational to algorithms such as REINFORCE, utilizes the log-derivative trick to express the gradient entirely in terms of expectations that can be estimated through sampling.[10] The gradient is mathematically formulated as:

\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left

10

While this Monte Carlo estimator is mathematically unbiased, it suffers from extreme variance because the cumulative return G_t depends on a long sequence of stochastic actions and environmental transitions.[10] To execute variance reduction without introducing bias, a

state-dependent baseline $b(s_t)$ is subtracted from the return.[10] The theoretically optimal baseline that minimizes the variance of the policy gradient estimator is the State-Value Function $V^\pi(s_t)$ itself.[10]

This subtraction yields the Advantage Function: $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$.[10] The Advantage function elegantly isolates the specific impact of an action, indicating whether taking action a in state s yields a return better or worse than the average return expected from simply arriving in state s.[10] Consequently, the variance-reduced policy gradient becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[A^\pi(s_t, a_t)\nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

[10]

This actor-critic formulation, where one network (the actor) learns the policy gradient while another network (the critic) learns the value function baseline, forms the theoretical bedrock for Proximal Policy Optimization (PPO), which will govern the online learning phase of the proposed ViZDoom architecture.[5]

3. The ViZDoom Environmental Ecosystem

The realization of the theoretical reinforcement learning frameworks requires a highly robust, high-throughput simulation engine. The ViZDoom platform provides unparalleled access to the internal mechanics of a 3D shooter, engineered specifically for machine visual learning.[2] Maintained for cross-platform compatibility across Linux, macOS, and Windows, the engine is capable of executing up to 7,000 frames per second in synchronous mode on a single thread.[2] This massive throughput is critical for deep reinforcement learning, where sample efficiency is low and agents typically require millions of environmental interactions to converge upon a stable policy.[2]

3.1. Observation Modalities and Environmental Buffers

Unlike early benchmarking environments that restricted agents to raw RGB arrays, ViZDoom operates under the paradigm that embodied intelligence requires multiple sensory modalities. The platform exposes a suite of parallel rendering buffers, effectively providing the agent with synthesized geometric and semantic data alongside raw visual input.[1]

Buffer Type	Data Description	Architectural Application

		and Cognitive Implication
Screen Buffer	Raw RGB24 or grayscale pixel matrices.	Simulates human-like visual processing. Serves as the primary, highest-dimensional input for CNN feature extractors, requiring the network to autonomously deduce lighting, enemy presence, and complex architectural features.[1]
Depth Buffer	High-precision 16-bit Z-buffer geometry visualization.	Grants the agent immediate, computationally inexpensive access to scene geometry. This facilitates the development of visual navigation, obstacle avoidance, and pseudo-SLAM (Simultaneous Localization and Mapping) behaviors without the massive overhead of external photogrammetry estimators.[1]
Labels Buffer	Semantic segmentation masks.	Provides a ground-truth, pixel-perfect categorization of game objects. This enables targeted supervised learning of object detection and is vital during the behavioral cloning phase to isolate critical entities (e.g., distinguishing an enemy from an inert decorative object).[1]

Audio Buffer	Raw sound wave data arrays.	Supports advanced multi-modal fusion research. Sound cues are critical for locating out-of-sight threats, allowing the agent to react to events occurring outside its immediate field of view.[1]
Automap	Top-down cartographic map data.	Typically utilized as "privileged information" in complex Teacher-Student network models or for debugging macroscopic navigation policies.[1]
Game Variables	Scalar internal engine states (Health, Ammo).	Essential for defining the agent's internal state vector and for constructing complex, multi-tiered reward shaping functions during RL optimization.[1]

For the specific architecture defined in this report, the environment is configured via YAML specification files to enforce a strict resolution of 320x240 pixels.[10] The engine processes the simulation at 35 tics per second (35 Hz). By setting the frame_skip parameter to 1, the architecture ensures that every single frame is evaluated during both human data collection and algorithmic inference, allowing the model to develop high-frequency reaction capabilities and continuous tracking behaviors.[10]

3.2. Action Space Topography

The physical translation of human intent into the agent's mathematical action space requires a structured discretization. The keymap.yaml file explicitly maps physical keystrokes into a strict 17-dimensional binary action vector that the neural network will output.[10]

Action Category	Binary Action Identifier	Real-World Key Mapping	Tactical Function
Locomotion	MOVE_FORWARD, MOVE_BACKWARD,	W, S, A, D	Controls absolute planar positioning

	MOVE_LEFT, MOVE_RIGHT		and enables evasive strafing maneuvers.[10]
Rotation	TURN_LEFT, TURN_RIGHT	Left Arrow, Right Arrow	Controls the agent's yaw, updating the field of view.[10]
Interaction	ATTACK, USE, SPEED	Space, E, Shift	Dictates offensive output, environmental manipulation (doors, switches), and velocity scaling.[10]
Inventory	SELECT_WEAPON1 through SELECT_WEAPON7	Number Row 1 - 7	Enables dynamic resource management based on enemy proximity and ammunition availability.[10]

During the human recording phase, the continuous nature of human mouse movements presents a challenge. While some architectures formulate mouse tracking as a continuous regression problem due to the limitations of one-hot encoding [3], the proposed architecture standardizes on the discrete binary vector approach. This ensures strict mathematical parity between the data collected during human gameplay and the discrete probability distributions evaluated by the Proximal Policy Optimization algorithms in the subsequent reinforcement learning phase.[5]

3.3. Reward Shaping Structure

The reward function $\mathcal{R}(s, a)$ is the invisible hand that guides the gradient ascent during the reinforcement learning phase. Relying solely on a sparse terminal reward (e.g., winning or losing the level) creates an impossibly vast search space for an untrained agent. Therefore, the architecture relies on dense reward shaping configured within the game_config.yaml file to provide intermediate optimization signals.[10]

Reward Trigger	Scalar Value	Behavioral Implication
Living Penalty	-0.02 per tic	Accumulates over time, applying constant temporal pressure to discourage passive, stationary survival strategies and force level completion.[10]
Death Penalty	-500.0	A catastrophic sparse penalty that generates a massive negative advantage, strongly discouraging terminal failure states.[10]
Kill Reward	+8.0	Shapes aggressive behavior, teaching the agent that eliminating threats yields positive returns.[10]
Health Pickup	+12.0	Normalizes self-preservation behavior. The high value encourages the agent to prioritize navigation toward medical resources when damaged.[10]
Armor Pickup	+5.0	Encourages proactive defensive resource gathering.[10]
Level Clear Bonus	+250.0	The ultimate sparse reward signifying the successful execution of a complete episodic trajectory.[10]

4. Algorithmic Frameworks and Competitive Intelligence

When selecting the foundational software library to orchestrate the neural networks and reinforcement learning loops, three primary frameworks dominate the ViZDoom research ecosystem: Sample Factory, Tianshou, and Stable Baselines 3.[1]

Framework	Architecture Type	Throughput (Approx. FPS)	Primary Research Application
Sample Factory	Asynchronous Actor-Learner	>100,000	Designed for massive-scale training and achieving State-of-the-Art performance in competitive environments. Maximizes hardware utilization through highly tuned asynchronous execution.[1]
Tianshou + EnvPool	Modular / C++ Batched	~20,000 - 50,000	Integrates with EnvPool to bypass Python overhead entirely. Favored for custom algorithm research and high-performance synchronous training.[1]
Stable-Baselines3 (SB3)	Synchronous Vectorized	~2,000 - 10,000	The industry standard for prototyping, education, and algorithmic baselines. While utilizing slower synchronous vectorized

			environments (DummyVecEnv or SubprocVecEnv), its strict adherence to mathematical standards and superior code readability make it ideal for complex, multi-stage hybrid pipelines involving imitation learning.[1]

For the architecture described in this report, Stable Baselines 3 is selected as the primary engine. Despite its lower raw throughput compared to Sample Factory, SB3's modular policy definitions, native support for Multi-Input dictionaries, and seamless integration with the imitation library render it the optimal choice for bridging Behavioral Cloning with PPO.[1]

Furthermore, an analysis of specialized implementations, such as the stable-baselines-agent from the ViZDoomBot repository, reveals several advanced architectural patterns that inform our design.[16] While default SB3 relies on the standard nature_cnn architecture, enhanced performance in Doom often requires custom, larger network topologies.[16] Advanced iterations of these agents frequently incorporate Long Short-Term Memory (LSTM) modules directly into the policy network to provide historical context, addressing the partial observability inherent in the game.[16] Additionally, these implementations often utilize Intrinsic Curiosity Modules (ICM) or Random Network Distillation (RND) to generate intrinsic rewards, solving the exploration problem in maps with highly sparse extrinsic rewards.[12]

5. High-Fidelity Human Gameplay Acquisition and Data Engineering

The initial phase of the architectural pipeline requires the acquisition of expert human demonstrations to facilitate Behavioral Cloning. Relying on the established ViZDoom engine, human gameplay is recorded. However, capturing multiple high-resolution buffers (RGB, Depth, Labels) simultaneously at 35 frames per second alongside scalar metadata introduces severe disk Input/Output (I/O) bottlenecks that can easily stall the game's internal tick rate, destroying the fidelity of the human recording.[3]

To circumvent this hardware limitation, the recording mechanism (typically housed in a module like doom_play.py) must employ a highly optimized, asynchronous multiprocessing architecture.[10]

5.1. Asynchronous Recording Architecture

During a recording session, human input is captured globally via libraries such as pynput. The system normalizes physical keystrokes into standardized string aliases and continuously maps them against the configurations to generate the 17-dimensional binary NumPy action vector for every single frame.[10]

Simultaneously, the main execution thread pulls the multi-modal visual buffers from the engine controller alongside the precise game variables, reward deltas, and episodic terminal flags.[10] Rather than executing blocking write operations to the hard drive, this bundled dictionary of state-action metadata is immediately pushed into a non-blocking multiprocessing.Queue.[10]

A dedicated backend writer process (_writer_process) continuously consumes this queue in the background, carefully managing data storage across varying formats optimized for specific data types [10]:

1. **RGB Video Optimization**: The 3-channel RGB array representing the screen buffer is converted to the BGR24 format. It is then piped via standard input (stdin) to an FFMPEG subprocess.[10] FFMPEG encodes the continuous stream utilizing the highly efficient libx264 codec, packaging it within a .mkv container. Applying a Constant Rate Factor (CRF) of 18 ensures visually lossless compression while maintaining minimal file sizes and preventing disk saturation.[10]
2. **Depth and Label Compression**: The depth and label buffers contain single-channel, high-precision 16-bit integers representing exact distances and categorical IDs.[10] These matrices cannot survive standard lossy video compression without destroying their mathematical integrity. Therefore, the backend process aggregates these matrices into temporary memory chunks.[10] Upon reaching a predefined threshold (e.g., 350 frames), the chunk is flushed to disk as a highly compressed NumPy archive (.npz).[10] This chunking methodology prevents random access memory (RAM) overflows during extended recording sessions.[10]
3. **Telemetry and Metadata Storage**: The precise step-by-step records—containing the binary action vectors, scalar game variables, and immediate step rewards—are written to a .parquet file utilizing columnar storage libraries such as Pandas.[10] The Parquet format provides vast improvements in read-speeds during the subsequent training phase compared to raw JSON or CSV formats.[10]

5.2. Data Transformation for the Imitation Library

The raw outputs (.mkv, .npz, .parquet) generated by the recording suite are highly efficient for storage but are not natively compatible with the SB3-adjacent imitation library. The imitation learning framework requires expert data to be explicitly structured as Trajectory or Transitions Python objects.[17]

A custom data engineering pipeline is required to parse the recorded session directories and synthesize the formats.[18] The process involves using OpenCV (cv2.VideoCapture) to extract the RGB frames from the .mkv container, ensuring exact chronological alignment with the telemetry data.[18] Simultaneously, the chunked .npz arrays are loaded to extract the depth and label matrices. These matrices are mathematically expanded and stacked channel-wise alongside the RGB data to form a unified, composite observation tensor for each timestep.[3]

The .parquet file is queried to extract the synchronized arrays of actions, rewards, and informational game variables.[19] These synchronized elements are packed into a list of imitation.data.types.TrajectoryWithRew dataclasses. In this structure, obs represents the full sequence of multi-modal tensors, acts represents the human actions, and terminal denotes the episode boundary.[17] Finally, to prepare the data for supervised batch learning, the imitation.data.rollout.flatten_trajectories() utility is applied. This function breaks the sequential trajectories down into individual, independent Transitions, creating a vast dataset of (s_t, a_t) observation-action pairs ideal for behavioral cloning.[17]

6. Multi-Modal Neural Network Architecture

With the high-fidelity expert dataset successfully preprocessed into transitions, the architectural focus shifts to defining the neural network policy capable of digesting the high-dimensional, multi-buffer inputs. Stable Baselines 3 provides the MultiInputPolicy, which natively handles dictionaries of observation spaces via the gym.spaces.Dict structure.[7] However, SB3's default internal CombinedExtractor automatically applies the standard Nature Atari CNN to any image it detects. While sufficient for basic RGB grids, this default architecture is highly suboptimal for processing disparate modalities like strict geometric depth and categorical semantic labels simultaneously.[7]

6.1. Custom Feature Extractor Design

To extract meaningful features from the RGB, Depth, and Label buffers independently before performing sensor fusion, the architecture must define a custom PyTorch class that inherits directly from stable_baselines3.common.torch_layers.BaseFeaturesExtractor.[7]

Within the __init__ constructor of this custom class, the extractor receives the full observation_space dictionary.[7] A PyTorch nn.ModuleDict is initialized to securely store independent computational graphs (sub-networks) for each specific input modality, ensuring that PyTorch accurately tracks the gradients for all parameters.[7]

1. **RGB Processing Stream**: The RGB tensor, possessing a shape of $(3, 240, 320)$, requires a deep Convolutional Neural Network to extract textures, lighting gradients, and complex entity recognition.[1] Inspired by the architecture detailed by Mnih et al., a custom CNN sequence featuring cascading Conv2d layers is deployed.[23] These layers utilize

Exponential Linear Unit (ELU) non-linearities, progressively downsampling the spatial resolution while increasing the channel depth, eventually flattening the output and passing it through dense layers to produce a 512-dimensional latent vector.[7]

2. **Depth Processing Stream**: The depth tensor $(1, 240, 320)$ represents absolute raw geometry.[1] Because depth buffers lack texture variance and contain smooth gradients, applying deep, complex convolutions is computationally wasteful. Instead, a shallower CNN—or even aggressive MaxPool2d spatial downsampling—is entirely sufficient to extract obstacle proximity boundaries. This stream reduces computational latency and flattens the output into a concise 128-dimensional vector.[7]
3. **Labels Processing Stream**: Semantic labels $(C, 240, 320)$ exist as highly structured categorical masks. Depending on the sparsity and cardinality of the labels, they can be processed via lightweight depthwise convolutions or flattened directly through linear projections, generating another discrete latent vector.[7]
4. **Game Variables Processing Stream**: The scalar values representing health, ammunition, and weapon states bypass convolutional filters entirely. These normalized floats are passed through a simple multi-layer perceptron (MLP) to generate a state embedding.[21]

Once constructed, the total output dimensionality of all flattened tensors is mathematically aggregated and assigned to the mandatory self._features_dim attribute, satisfying the base class requirements.[7]

6.2. The Forward Pass and Sensor Fusion

During inference and training, the forward() method orchestrates the sensor fusion. The method iterates over the observations dictionary, routing each specific tensor through its corresponding, dedicated sub-network within the nn.ModuleDict.[7] The resulting latent vectors from the RGB, depth, label, and scalar streams are collected into a list.

These disparate representations are then fused mathematically using PyTorch's concatenation operation: torch.cat(encoded_tensor_list, dim=1).[7] This unified, multi-modal feature vector acts as the comprehensive state representation s_t. It is subsequently passed into the policy's standard fully-connected layers, defined by the net_arch parameter, which splits the representation into the Actor head (predicting action probabilities) and the Critic head (predicting the state-value).[7]

This custom extractor is injected directly into the learning algorithms by passing the class definition through the initialization arguments: policy_kwargs = dict(features_extractor_class=CustomMultiImageExtractor).[7] Furthermore, for on-policy algorithms like PPO, this feature extractor is shared by default between the actor and the critic to drastically reduce computational load, though off-policy algorithms generally perform better

with completely separate feature extractors.[7]

7. Phase I: Pre-training via Behavioral Cloning (BC)

Attempting to train a standard reinforcement learning agent from scratch in a complex 3D environment featuring both sparse and dense rewards inevitably results in millions of frames of erratic, random behavior. An untrained agent will likely spin in circles, fire at walls, and suffer repeated -500 death penalties before ever accidentally discovering the complex sequence of actions required to trigger a positive reward signal.[10] This severe sample inefficiency is drastically mitigated by applying Imitation Learning (IL), specifically Behavioral Cloning.[3]

7.1. The Behavioral Cloning Framework

Behavioral Cloning treats the complex problem of sequential decision-making as a pure supervised learning task.[25] Utilizing the massive dataset of Transitions generated from the human gameplay recordings, the neural network learns to reproduce the expert's behavior directly.[25] For a given state observation s_t, the policy network π_θ is trained such that its predicted action distribution minimizes the error against the ground-truth action a_t taken by the human expert.[25]

The imitation.algorithms.bc.BC class serves as the robust engine for this phase.[27] The custom multi-input SB3 policy architecture defined previously is instantiated and passed directly into the BC constructor, alongside the observation and action spaces derived from the environment.[27]

7.2. Training Mechanics and Optimization

The BC training loop processes the observation-action pairs in minibatches. Because the input space consists of multiple high-resolution image tensors, memory management on the GPU becomes a critical bottleneck.[27] The imitation library's BC module elegantly solves this by allowing developers to specify a batch_size alongside a smaller minibatch_size. This architecture facilitates gradient accumulation over the smaller chunks, effectively mimicking a massive batch size to stabilize gradients without triggering catastrophic Out-Of-Memory (OOM) exceptions on the hardware.[27]

Optimization relies on standard stochastic gradient descent variants, defaulting to the Adam optimizer (torch.optim.Adam).[27] Because the architecture maps the 17 game buttons to a multi-discrete or multi-binary distribution, the loss function evaluates a form of cross-entropy loss against the human actions, penalizing the network for deviating from the expert's choices.[3]

The training script invokes the process via bc_trainer.train(n_epochs=X).[27] A critical danger in

behavioral cloning is overfitting, wherein the policy rigidly memorizes the exact training frames rather than learning generalized, causal heuristics. To combat this, the framework allows for the strict application of L2 regularization via the l2_weight parameter, and entropy regularization via the ent_weight parameter.[27] Entropy regularization is specifically vital; it prevents the action probability distribution from collapsing into a completely deterministic delta function too early. By preserving a slight degree of stochasticity within the predictions, the network remains flexible, which is an absolute necessity for the subsequent reinforcement learning phase where exploration is required.[10]

7.3. The Problem of Covariate Shift

While Behavioral Cloning is remarkably fast at instilling basic navigational competence and visual recognition, it suffers fundamentally from covariate shift, frequently referred to in literature as compounding errors.[26] Because the agent is trained exclusively on states visited by an expert human player, the dataset is inherently biased; it contains no examples of the agent making catastrophic mistakes.[26]

Consequently, if the BC-trained agent makes a slight navigational error during actual gameplay that pushes it into an out-of-distribution state (e.g., getting wedged in a corner or staring directly at a blank wall), the neural network has no training data regarding how to recover. The human expert never got stuck in the corner, so the dataset contains no examples of corrective maneuvers. Without corrective data, the agent's errors compound rapidly, leading to complete systemic failure.[26] Therefore, Behavioral Cloning cannot be the final step for a robust Doom agent. It serves solely to seed the neural network weights with a foundational understanding of the environment's visual topology and rudimentary action-causality.[1]

8. Phase II: Online Reinforcement Learning via Proximal Policy Optimization (PPO)

To overcome the fragility and covariate shift inherent in the BC-trained agent, the model must transition to an active, online reinforcement learning phase. In this phase, the agent interacts directly with the ViZDoom environment in real-time. Through trial and error, guided by the complex, multi-tiered reward structure defined in game_config.yaml, the agent explores the environment autonomously, experiences out-of-distribution states, and learns corrective behaviors to maximize its Expected Cumulative Return.[10]

The chosen algorithm for this online fine-tuning phase is Proximal Policy Optimization (PPO), a state-of-the-art policy gradient method universally lauded for balancing exceptional sample efficiency with robust training stability, making it ideal for continuous, high-dimensional control tasks.[5]

8.1. Theoretical Mechanics of Proximal Policy

Optimization

PPO builds directly upon the mathematical foundations of Trust Region Policy Optimization (TRPO). Traditional policy gradients can experience catastrophic performance degradation if a single, aggressive update step moves the network weights too far in the wrong direction, instantly destroying a functional policy.[10] TRPO solves this by enforcing a hard Kullback-Leibler (KL) divergence constraint between the old and new policies, but this requires complex, computationally prohibitive second-order derivative calculations involving the Fisher Information Matrix and conjugate gradients.[10]

PPO brilliantly approximates this trust region mathematically without the computational overhead by utilizing a Clipped Surrogate Objective.[5] Let $r_\theta(s,a)$ denote the probability ratio between the newly updated policy π_θ and the old policy π_{old}, defined as $r_\theta(s,a) = \frac{\pi_\theta(a|s)}{\pi_{old}(a|s)}$.[10] The core PPO clipped objective function is defined mathematically as:

$$\mathcal{L}^{clip}(\theta) = \mathbb{E}_{(s,a)\sim\pi_{old}}\left[\min\left(r_\theta(s,a)A^{old}(s,a), \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)A^{old}(s,a)\right)\right]$$

[10]

The clip function strictly restricts the ratio r_θ to the interval $[1-\epsilon, 1+\epsilon]$, where the hyperparameter ϵ is typically set to 0.2.[10] This creates a conservative update mechanism. If the estimated advantage $A^{old}(s,a) > 0$, indicating the action was unexpectedly good, PPO encourages increasing the probability of that action. However, the clipping function prevents the probability from skyrocketing uncontrollably.[10] Conversely, if $A^{old}(s,a) < 0$, indicating a poor action, PPO reduces the action's probability, but the clipping prevents it from plummeting to zero instantly, ensuring the policy does not collapse.[10]

The total PPO loss function incorporates two additional mathematical terms to simultaneously train the critic network and ensure continued environmental exploration [10]:

$$\mathcal{L}^{PPO}(\theta) = \mathcal{L}^{clip}(\theta) - c_1\mathcal{L}_V(\theta) + c_2\mathbb{E}\left[\mathcal{H}(\pi_\theta(\cdot|s))\right]$$

[10]

The Value Loss ($\mathcal{L}_V$) utilizes a Mean Squared Error (MSE) formulation to train the critic network to accurately predict the State-Value $V_\theta(s)$, which is absolutely necessary to compute the

Advantage estimations accurately.[10] The Entropy Bonus ($\mathcal{H}$) rewards the policy for maintaining a level of statistical randomness, calculated as $\mathcal{H}(\pi_\theta(\cdot|s)) = -\sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$.[10] This explicitly forces the agent to continually experiment with new tactics—such as trying different weapons or complex strafing patterns—rather than prematurely converging on a safe, but suboptimal, local minimum.[10]

8.2. Model Surgery and Weight Transfer Implementation

The transition from the imitation library's Behavioral Cloning trainer to the Stable Baselines 3 PPO instance presents a specific and well-documented software engineering hurdle. While both libraries ultimately rely on the same underlying PyTorch architecture (SB3's ActorCriticPolicy), they initialize specific internal sub-components differently.[31]

Attempting to pass the BC-trained policy directly into the PPO constructor as an argument (i.e., model = PPO(policy=bc_trainer.policy, env=env)) results in a catastrophic TypeError regarding the unexpected keyword argument use_sde (State Dependent Exploration).[31] The traceback confirms this error occurs deep within Stable Baselines 3's _setup_model() initialization sequence, specifically when attempting to invoke the policy's forward() method.[31] This indicates a structural mismatch in the internal network configurations expected by the algorithm.[31]

To ensure a flawless transfer of the learned visual representations and behavioral heuristics without triggering framework errors, the architecture employs a PyTorch state-dictionary injection, commonly referred to as Model Surgery.[32] The methodology is implemented through the following steps:

1. **Initialize the Untrained PPO Model**: The PPO model is instantiated normally, from scratch, using the custom MultiInputPolicy and the CustomMultiImageExtractor previously defined via the policy_kwargs argument.[7]
2. **Extract BC State Dictionary**: The raw PyTorch parameters—representing the learned weights and biases from the human data—are extracted from the BC policy using the state_dict() method: bc_weights = bc_trainer.policy.state_dict().[32]
3. **Inject Weights into PPO**: The extracted weights are systematically loaded into the newly initialized PPO policy network via ppo_model.policy.load_state_dict(bc_weights, strict=False).[32]

Setting the strict=False parameter during the dictionary load is a crucial engineering decision. It ensures that if there are minor mismatches in auxiliary nodes—such as the standard deviation logs used internally by PPO's stochastic exploration mechanism that the deterministic BC process might lack—the core convolutional feature extractor layers and the Multi-Layer

Perceptron actor/critic heads are still correctly transplanted.[32]

8.3. Online Fine-Tuning Execution

Once the pre-trained weights are successfully loaded via model surgery, the agent commences active interaction with the simulated environments. To maximize throughput, the architecture utilizes make_vec_env to run several independent instances of Doom simultaneously, gathering massive amounts of experiential data rapidly across multiple CPU cores.[30]

During this critical online phase, the pre-trained visual hierarchy instantly recognizes geometric boundaries from the depth buffer and semantic enemy locations from the label buffer. This entirely bypasses the millions of steps normally required for an RL agent to learn basic convolutional edge detection and feature extraction.[1] The PPO algorithm, operating on these advanced representations, strictly focuses on mapping actions against the defined -500 and $+250$ reward thresholds.[10] Through iterative optimization of the clipped surrogate objective, PPO refines the agent's combat heuristics, corrects the covariate shift vulnerabilities, and develops complex strafing logic to achieve robust, near-optimal play within the ViZDoom environment.[1]

9. Advanced Methodologies and Future Architectural Augmentations

While the integration of multi-modal CNNs with BC and PPO creates a formidable agent, pushing performance toward state-of-the-art benchmarks in highly complex "Full Deathmatch" tracks requires further architectural augmentations.[1]

The primary limitation of standard CNN-based PPO architectures is the lack of temporal memory. A strictly feed-forward network evaluates each frame in isolation, leaving the agent vulnerable to partial observability (e.g., forgetting an enemy is hiding behind a pillar once they break line of sight).[1] To rectify this, advanced architectures integrate Long Short-Term Memory (LSTM) cells directly into the policy network.[2] Alternatively, cutting-edge research has demonstrated the efficacy of replacing recurrent models entirely with Transformer-based architectures, such as the Online Decision Transformer (ODT) or UPDeT.[38] These transformer models process entire sequences of agent observations simultaneously using self-attention mechanisms, excelling at managing long-horizon temporal dependencies and exhibiting capabilities for sophisticated tactical decision-making.[39]

Furthermore, architectural enhancements like the Recurrent Implicit Sequential Extraction (RISE) framework offer methods to utilize recurrent models with significantly reduced computational overhead compared to traditional LSTM implementations.[40] These

advancements, when coupled with the foundational Behavioral Cloning and Proximal Policy Optimization pipeline detailed in this report, provide the necessary trajectory to develop highly adaptive, generalized agents capable of mastering the complexities of three-dimensional embodied perception.

Works cited

1. Advanced Architectures and Methodologies in Visual Reinforcement Learning: A Technical Analysis of the ViZDoom Platform - Finding Theta, accessed March 2, 2026, https://findingtheta.com/blog/advanced-architectures-and-methodologies-in-visual-reinforcement-learning-a-technical-analysis-of-the-vizdoom-platform
2. Farama-Foundation/ViZDoom: Reinforcement Learning environments based on the 1993 game Doom :godmode - GitHub, accessed March 2, 2026, https://github.com/Farama-Foundation/ViZDoom
3. [Literature Review] Behavioural Cloning in VizDoom - Moonlight, accessed March 2, 2026, https://www.themoonlight.io/en/review/behavioural-cloning-in-vizdoom
4. Behavioural Cloning in VizDoom, accessed March 2, 2026, https://arxiv.org/abs/2401.03993
5. PPO — Stable Baselines3 2.8.0a4 documentation, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html
6. ViZDoom performance. "depth" means generating also the depth buffer. - ResearchGate, accessed March 2, 2026, https://www.researchgate.net/figure/ZDoom-performance-depth-means-generating-also-the-depth-buffer_fig6_302567940
7. Policy Networks — Stable Baselines3 2.8.0a4 documentation - Read the Docs, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html
8. Train a RL model on the video game Doom - GitHub, accessed March 2, 2026, https://github.com/GuillBla/RL-Doom
9. Behavioural Cloning in VizDoom - Sony Interactive Entertainment, accessed March 2, 2026, https://sonyinteractive.com/en/innovation/research-academia/research/behavioural-cloning-in-vizdoom/
10. Session7.0_Reinforcement learning.pdf
11. FAQ - ViZDoom Documentation, accessed March 2, 2026, https://vizdoom.farama.org/faq/index.html
12. callumhay/vizdoom_ppo_rnd: A doom agent that can beat levels in vizdoom using a combination of Proximal Policy Optimization (PPO) and Random Network Distillation (RND). - GitHub, accessed March 2, 2026, https://github.com/callumhay/vizdoom_ppo_rnd
13. Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations — Stable Baselines3 2.8.0a4 documentation, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/
14. Imitation Learning — Stable Baselines3 1.4.0 documentation, accessed March 2,

2026, https://stable-baselines3.readthedocs.io/en/v1.4.0/guide/imitation.html
15. Imitation Learning — Stable Baselines3 2.8.0a4 documentation - Read the Docs, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/master/guide/imitation.html
16. ViZDoomBot/stable-baselines-agent - GitHub, accessed March 2, 2026, https://github.com/ViZDoomBot/stable-baselines-agent
17. Trajectories - imitation - Read the Docs, accessed March 2, 2026, https://imitation.readthedocs.io/en/latest/main-concepts/trajectories.html
18. lmangani/videoparquet: Python library for converting Parquet files (containing array-like or tabular data) to video files and back using ffmpeg - GitHub, accessed March 2, 2026, https://github.com/lmangani/videoparquet
19. Create Parquet files from stream in python in memory-efficient manner - Stack Overflow, accessed March 2, 2026, https://stackoverflow.com/questions/64791558/create-parquet-files-from-stream-in-python-in-memory-efficient-manner
20. Train an Agent using Behavior Cloning - imitation, accessed March 2, 2026, https://imitation.readthedocs.io/en/stable/tutorials/1_train_bc.html
21. Policy Networks — Stable Baselines3 2.8.0a4 documentation, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html#custom-network-architecture
22. iGibson/igibson/examples/learning/stable_baselines3_example.py at master - GitHub, accessed March 2, 2026, https://github.com/StanfordVL/iGibson/blob/master/igibson/examples/learning/stable_baselines3_example.py
23. HASARD: A Benchmark for Vision-Based Safe Reinforcement Learning in Embodied Agents, accessed March 2, 2026, https://arxiv.org/html/2503.08241v1
24. Is Behavior Cloning All You Need? Understanding Horizon in Imitation Learning - NeurIPS, accessed March 2, 2026, https://neurips.cc/virtual/2024/poster/96376
25. Pre-Training (Behavior Cloning) — Stable Baselines 2.10.3a0 documentation, accessed March 2, 2026, https://stable-baselines.readthedocs.io/en/master/guide/pretrain.html
26. Imitation Learning Algorithms Tutorial (Python) from scratch - GitHub, accessed March 2, 2026, https://github.com/tsmatz/imitation-learning-tutorials
27. Behavioral Cloning (BC) - imitation, accessed March 2, 2026, https://imitation.readthedocs.io/en/latest/algorithms/bc.html
28. Introduction to Imitation Learning & the Behavior Cloning Algorithm - Wen Sun, accessed March 2, 2026, https://wensun.github.io/CS4789_data/Imitation_Learning_April_8_annotated.pdf
29. Behavioral Cloning (BC) - imitation - Read the Docs, accessed March 2, 2026, https://imitation.readthedocs.io/en/stable/algorithms/bc.html
30. PPO — Stable Baselines3 0.8.0 documentation - Read the Docs, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/sde/modules/ppo.html
31. Got an unexpected keyword argument 'use_sde' when passing ..., accessed March 2, 2026, https://github.com/HumanCompatibleAI/imitation/issues/781

32. Transfer learning in reinforcement learning environment for different observation and action spaces - PyTorch Forums, accessed March 2, 2026, https://discuss.pytorch.org/t/transfer-learning-in-reinforcement-learning-environment-for-different-observation-and-action-spaces/171011
33. On saving and loading — Stable Baselines3 2.8.0a4 documentation, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/master/guide/save_format.html
34. Custom Environment Trajectory gathering · Issue #281 ... - GitHub, accessed March 2, 2026, https://github.com/HumanCompatibleAI/imitation/issues/281
35. Training a Custom Feature Extractor in Stable Baselines3 Starting from Pre-trained Weights? - Stack Overflow, accessed March 2, 2026, https://stackoverflow.com/questions/78718554/training-a-custom-feature-extractor-in-stable-baselines3-starting-from-pre-train
36. PPO — Stable Baselines3 2.4.0 documentation, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/v2.4.0/modules/ppo.html
37. PPO — Stable Baselines3 1.0 documentation - Read the Docs, accessed March 2, 2026, https://stable-baselines3.readthedocs.io/en/v1.0/modules/ppo.html
38. FAST(ER) DATA GENERATION FOR OFFLINE RL AND FPS ENVIRONMENTS FOR DECISION TRANSFORMERS - Purdue University Graduate School, accessed March 2, 2026, https://hammer.purdue.edu/ndownloader/files/43477251
39. Deep Reinforcement Learning-Based Multi-Agent System with Advanced Actor–Critic Framework for Complex Environment - ResearchGate, accessed March 2, 2026, https://www.researchgate.net/publication/389339264_Deep_Reinforcement_Learning-Based_Multi-Agent_System_with_Advanced_Actor-Critic_Framework_for_Complex_Environment
40. Recurrent Off-Policy Deep Reinforcement Learning Doesn't Have to be Slow - arXiv, accessed March 2, 2026, https://arxiv.org/html/2512.20513v1